

Collectors Village

Platform Audit Report

Security · Code Quality · User Experience · Administrative Efficiency

Prepared: July 19, 2026

Scope: Django/DRF backend, Next.js frontend, and the admin/vendor/customer experience.

Out of scope (by request): the Set Registry feature.

This report reviews the codebase end to end and translates the technical findings into plain-language priorities: what could go wrong, what's already working well, what's missing for a polished public launch, and concrete ways to make the admin side faster to run day to day.

Executive Summary

The short version: the foundation is solid. Vendor/admin/customer roles are enforced correctly everywhere we checked, sensitive data (prices, contact info, emails) is kept out of public API responses, and the production settings already do the right things around HTTPS and secure cookies. There is no evidence of careless or reckless coding anywhere in the app.

That said, this is still very much a working prototype rather than a finished, public-ready marketplace. The most important thing to understand before launch: **the homepage, Browse Vendors, and Browse Cards pages are still showing sample/placeholder vendors and inventory, not real data from actual sellers.** A real vendor who signs up, gets approved, and lists inventory today will not show up on those pages yet — only on their own vendor dashboard and the events/venue tools. This is the single biggest gap standing between where the app is now and a real public launch.

Top 5 priorities before a public launch

- **Connect real vendor/listing data to the public browse pages** — homepage, Browse Vendors, Browse Cards currently show fake sample data instead of real sellers and inventory.
- **Add basic protection against automated abuse** — login, signup, and password reset have no rate limiting today, so a script could hammer them.
- **Move login sessions out of browser storage** — a known, already-flagged shortcut that should be closed before real customer accounts are on the platform.
- **Fix venue floor-plan image uploads for the real (non-local) server** — as configured, uploaded floor plan images would not actually appear once the app is deployed off a developer's own machine.
- **Make the floor-plan editor usable on tablets and by keyboard** — right now it only works with a mouse, which matters if admins ever want to build a floor plan from an iPad at a venue.

None of these require a rewrite. Each is a well-scoped, bounded piece of work — details and effort estimates are in the sections that follow, with a single prioritized list at the end of this report.

A. Security Findings

"Security" findings below cover anything that could let someone do something they shouldn't — see private data, take over an account, abuse a form, or take the site down. Severity reflects real-world impact, not how technical the issue sounds.

Severity	Finding	Why it matters	Recommended fix
HIGH	No protection against repeated login/signup/password-reset attempts.	A script can try thousands of passwords per minute against any account, or flood the signup form with fake accounts, with nothing in the system slowing it down.	Add rate limiting to those three endpoints (Django REST Framework has this built in — it's a settings change plus some tuning, not new code from scratch).
HIGH	Login sessions are stored in the browser's local storage rather than a more locked-down cookie.	This was already flagged in the code as a temporary shortcut. If any other bug ever let an attacker run a script on the page, they could steal a logged-in session and keep renewing it indefinitely.	Move to an httpOnly cookie set by the server (the code already documents the intended approach — it's planned, just not done yet).
HIGH	Uploaded venue floor-plan images only work on a developer's local machine, not on a real server.	As currently configured, once the app is deployed for real, admin-uploaded floor plan images would silently fail to display anywhere on the site.	Connect image uploads to a real file-storage service (e.g. a cloud storage bucket) before launch — this is a standard, well-understood step for any Django app going to production.
HIGH	The app's master security key falls back to a built-in placeholder if not explicitly configured.	If the production server is ever set up without this one setting, the app will still start up — just with a security key that's sitting in plain text in this very report and codebase.	Make the app refuse to start at all in production if this isn't explicitly set (a small, one-time settings change).
MEDIUM	A card listing's price can be set to a negative number if someone calls the API directly (bypassing the normal "Add Item" form).	The form itself won't let a vendor type a negative price, but the server doesn't double-check, so a negative price could theoretically slip through outside the normal screen.	Add the same "must be zero or more" rule on the server side that already exists for venue booth prices.
MEDIUM	New account emails aren't required to be verified before the account can be used.	Someone could sign up with an email address that isn't actually theirs and start using the account immediately.	Turn on "require email verification" for real launch (already supported, just not switched on).
MEDIUM	If the app is ever accidentally pointed at its "local development" settings instead of "production" settings, detailed error pages and open host access are on by default.	This is the classic way sites accidentally leak internal error details (file paths, code snippets) to the public internet.	Add a safety check that stops the app from starting if it's not clearly using the right settings for where it's running.
MEDIUM	No size or type limit is enforced on venue floor-plan image uploads beyond basic image checks.	An admin account (or one that's been compromised) could upload an unusually large file and slow down or fill up the server.	Add a straightforward file-size cap for uploads.

Severity	Finding	Why it matters	Recommended fix
LOW	A couple of supporting code libraries (image processing, encryption) are a version or so behind current.	Not an active problem today, but staying a version behind means missing out on bug fixes made upstream.	Routine dependency updates — a normal, low-drama maintenance task.
LOW	Password-reset emails are only ever printed to the server log right now, not actually emailed.	"Forgot password" doesn't yet send a real email to anyone — fine for development, not for real users.	Connect a real email-sending service before launch (needed anyway for vendor-approval notifications, see Section D).

What's already done well

- Every admin-only, vendor-only, and "my own data only" boundary we checked across the entire backend is enforced correctly — vendors can't see or edit each other's listings, customers can't reach admin tools, and none of that depends on the frontend hiding a button (it's enforced on the server, which is what matters).
- Public pages are careful about what they show: booth prices and vendor contact details never leak to the public floor-map view, and a hidden event page and a nonexistent one look identical from the outside (so no one can "probe" for hidden content).
- Approve/reject and confirm/decline actions are locked to their specific outcome by the button clicked, not a hidden field — so there's no way to trick the system into approving something by tampering with a request.
- Production settings already correctly force HTTPS, secure cookies, and standard browser security headers.
- No unsafe patterns anywhere in the code — no raw database queries built from user input, no unsafe HTML injection on the frontend, no hardcoded passwords or API keys checked into the codebase.

B. Code Quality & Reliability

Automated test coverage

The backend (the part handling data, accounts, payments-adjacent logic) has strong test coverage: 114+ automated tests covering signup, login, onboarding, vendor approval, events, venues, booth selection, and the loyalty-hold system, all currently passing. That's a genuinely good safety net — it means future changes are far less likely to silently break something that already worked.

The frontend (the part users actually click through) has **no automated tests at all** — no test runner is even installed. Every check today is manual (someone clicking through the site). That's normal at this stage of a project, but it's worth budgeting for before the app grows much further, since UI regressions otherwise only get caught by a person happening to notice.

Error handling

Several pages — Browse Events, the homepage, the admin Manage Events list, and the vendor's own inventory load — fail silently if the server has a hiccup. Instead of an error message, the visitor just sees an empty "no events" or "no items" screen, which looks identical to there genuinely being nothing there. This is an easy, low-risk fix (the pattern for showing a real error message already exists and works correctly on several other pages — it just needs to be applied consistently).

Dependency health

Most backend packages (Django, the API framework, authentication libraries) are current and well maintained. A couple of supporting libraries are a version behind (noted in Section A) — routine, low-effort maintenance.

C. User Experience, by Audience

The app already has genuinely good bones for all three audiences — the events/venue/booth system in particular is more sophisticated than most card-show sites in the wild. The gaps below are what stand between "a good tool" and "a good experience" for each group.

General public / customers

- **Biggest gap:** the homepage, Browse Vendors, and Browse Cards pages show sample/placeholder vendors and inventory instead of real ones. A visitor browsing today isn't seeing what's actually for sale.
- There are actually two "vendor profile" pages right now — a real, working one and an older placeholder one — and browsing currently sends visitors to the placeholder. This will sort itself out as part of connecting real data above.
- The "My Messages" area on a customer's dashboard is also just sample content — there's no real messaging between customers and vendors yet.
- When Browse Events can't reach the server, a visitor just sees "no upcoming events" with no indication anything went wrong — worth a proper "something went wrong, try again" message instead.

Vendors

- No email notifications exist anywhere yet — a vendor who applies has to keep checking back to see if they've been approved, and one who requests a booth has to keep checking to see if it's been confirmed. A simple "you've been approved" / "your booth is confirmed" email would remove a lot of friction.
- Booth selection (the map where a vendor picks their spot) is mouse/click based, which works fine on a laptop but not as well on a phone — worth keeping in mind if vendors are expected to do this on the go.
- The listing price field can technically be pushed negative via a direct API call (see Security, Section A) — low real-world risk today, but worth closing before it matters.

Admins

This is the area that's improved the most recently — the booth-request queue now has a live count badge, an aggregated review page with a floor-plan preview for every pending request, and placing booths supports click-to-place with a remembered price/size/row so a large venue doesn't require retyping the same details hundreds of times. See Section D for the full picture and what's still worth adding.

- The floor-plan editor (where booths/sections get placed on the map) is mouse-only today — there's no keyboard way to place or adjust a booth, and it won't work at all on a tablet (no touch support). Fine on a laptop at a desk, not fine if an admin wants to build a floor plan from an iPad on-site at a venue.
- There's no audit trail of admin actions (who approved which vendor, who changed whose role) — useful once more than one person has admin access.
- The Manage Events and Manage Venues admin lists can silently fail to load on a server hiccup, same as the public pages above.

D. Speeding Up the Administrative Process

Already shipped this pass

- **One tool to see everything waiting on you:** the Admin Tools page now shows a live count badge on "Vendor Approvals" and a new "Booth Requests" tool, so there's no more clicking into each event to check for pending requests.
- **Booth Requests is now one page for the whole platform** instead of per-event — every pending request, from every event, in one place, each with a small floor-plan picture showing exactly which booth was requested, with Confirm/Decline right there.
- **Placing 200+ booths at a large venue is now dramatically faster:** set a price, a size, and a starting row letter once, then just click across the map — each click drops a booth with the next number in sequence (A001, A002, A003...) automatically, no repeated typing.
- **Venue search** for attaching a floor plan to an event now filters as you type instead of scrolling a long list — built with growth to 100+ venues in mind.

Recommended next

- **Bulk booth import from a spreadsheet.** Click-to-place is much faster than before, but for a venue with a precise, already-known layout (e.g. from a venue diagram), being able to upload a spreadsheet of booth numbers/prices/positions would beat even fast clicking.
- **"Clone this venue" for a new floor plan.** If two venues are similar (or the same venue gets a layout tweak year to year), starting from a copy of an existing floor plan instead of from scratch would save real time.
- **Bulk vendor approval.** Approving vendors one at a time is fine at low volume; a "select multiple, approve all" option would help once signups pick up.
- **Email notifications for admins, not just vendors.** "New vendor application" and "new booth request" emails would mean admins don't have to keep the dashboard open and refresh it to know something needs attention.
- **A simple admin activity log.** Once more than one admin exists, a basic "who approved/changed what, and when" log becomes valuable for accountability and troubleshooting.
- **A one-glance dashboard.** Booth occupancy rate for an upcoming event, vendor-approval turnaround time, and total booths confirmed vs. available — a small set of numbers at the top of the admin hub would give a sense of how a show is filling up without digging into each event.

E. Prioritized Roadmap

"Now" = worth doing before a real public launch with real customer/vendor accounts. "Next" = the following couple of sprints. "Later" = valuable but not urgent. Effort is a rough size, not a quote: S = a day or less, M = a few days, L = a week or more.

Priority	Item	Area	Effort
Now	Connect real vendor/listing data to homepage, Browse Vendors, Browse Cards	UX	M
Now	Add rate limiting to login/signup/password-reset	Security	S
Now	Move login sessions to an httpOnly cookie instead of browser storage	Security	M
Now	Fix venue floor-plan image uploads for real (non-local) hosting	Security	M
Now	Require the production security key to be explicitly set (no silent fallback)	Security	S
Next	Server-side floor on listing price (matches existing booth-price rule)	Security	S
Next	Turn on required email verification for new accounts	Security	S
Next	Guard against the app ever booting with local/dev settings by mistake	Security	S
Next	Add a file-size limit on floor-plan image uploads	Security	S
Next	Connect a real email service (password reset, vendor approval, booth confirmation)	UX	M
Next	Show a real error message instead of a silent empty state on API failures	UX	S
Next	Bulk booth import from a spreadsheet	Admin	M
Next	"Clone this venue" to reuse an existing floor plan	Admin	S
Later	Bulk vendor-approval actions	Admin	S
Later	Admin activity log (who approved/changed what)	Admin	M
Later	One-glance admin dashboard (occupancy, approval turnaround, revenue)	Admin	M
Later	Keyboard + touch support for the floor-plan editor	UX	L
Later	Real messaging between customers and vendors	UX	L
Later	Automated frontend test suite	Quality	L
Later	Routine dependency version bumps (image/encryption libraries)	Security	S

A note on scope

As requested, the Set Registry feature was left out of this review entirely and isn't reflected anywhere above. Everything else in the app — accounts, onboarding, events, venues, booths, listings, and all three admin/vendor/customer dashboards — was covered.